

UNIDAD 3 · SEMANAS 3-4

Archivos y sistema de archivos

📅 Semanas 3-4 · fechas por confirmar </> Laboratorio TD2 (resuelto + desafío IA)

~ 4 animaciones

MANUAL M3 · TEORÍA DE LA UNIDAD

Archivos y sistema de archivos

Lectura base de la unidad 3 (≈ 1.5 h). Cada sección termina con un enlace **Practica esto** →. Las salidas son reales: se obtuvieron con los programas del TD2 en el equipo del curso (Ubuntu 26.04, Linux 7.0, ext4 sobre NVMe).

CONTENIDO

- | | | | |
|----|---------------------------------------|----|---|
| §1 | Dos niveles: el disco y el núcleo | §5 | Archivos especiales: FIFO, dispositivos, <code>/proc</code> |
| §2 | El inodo: metadatos y bloques | §6 | Recorrer directorios |
| §3 | Nombres: directorios, enlaces y rutas | §7 | Más allá: diario, copia en escritura y <code>fsync</code> |
| §4 | Archivos abiertos: las tres tablas | | |

§1 Dos niveles: el disco y el núcleo

Un archivo existe en dos lugares. **En el disco**, el sistema de archivos organiza un enorme arreglo de bloques (de 4 KiB en el equipo del curso): un *superbloque* con la descripción general, mapas de bits que dicen qué bloques e inodos están libres, las *tablas de inodos* y los *bloques de datos*. ext4 repite esta estructura en *grupos de bloques* a lo largo de la partición, con copias del superbloque. El número de inodos se fija al formatear:

Salida real

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /
block size: 4096 bytes
```

En el núcleo, mientras un proceso usa el archivo, hay tres tablas: la de descriptores de cada proceso, la de archivos abiertos del sistema y la de inodos en memoria (§4). Encima de todo, la *caché de páginas* guarda en RAM los bloques leídos recientemente: por eso la segunda lectura de un archivo casi nunca toca el disco.

Míralo en la presentación P3 →

§2 El inodo: metadatos y bloques

Unix separa el **contenido** de un archivo de sus **metadatos**. Los metadatos viven en un **inodo**, identificado por un número: tipo y permisos (`st_mode`), número de enlaces duros (`st_nlink`), dueño y grupo, tamaño, dispositivo, marcas de tiempo y la lista de bloques. El inodo **no** contiene el nombre. `stat`, `fstat` (sobre un descriptor) y `lstat` (sin seguir enlaces) lo leen:

Salida real

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096   regular file
Device: 259,5      Inode: 12190686    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: ( 1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

Las cuatro marcas de tiempo: **Access** (última lectura; Linux la actualiza de forma perezosa con `relatime`), **Modify** (último cambio del contenido), **Change** (último cambio del *inodo*: permisos, enlaces, dueño... o contenido) y **Birth** (creación, disponible en ext4 y leída con `statx`). Es un error frecuente —las diapositivas originales lo cometían— creer que `ctime` es la creación.

El tipo se prueba con las macros `S_ISREG`, `S_ISDIR`, `S_ISLNK`, `S_ISCHR`, `S_ISBLK`, `S_ISFIFO` y `S_ISSOCK` sobre `st_mode`. Los errores típicos de `stat` son `ENOENT` (no existe), `EACCES` (falta permiso de paso `x` en un directorio de la ruta), `ENOTDIR` y `ELoop`.

¿Dónde están los bloques? ext2 y ext3 guardaban en el inodo 15 punteros: 12 directos, uno indirecto simple (un bloque lleno de punteros), uno doble y uno triple. Con bloques de 4 KiB y punteros de 4 bytes caben 1024 por bloque, y el esquema alcanza unos 4 TiB, pero una posición lejana exige leer hasta 3 bloques de índices. ext4 usa **extents**: tramos contiguos descritos por (inicio lógico, longitud $\leq 32\,768$ bloques, inicio físico). En el inodo caben 4; si hacen falta más, se forma un árbol. El archivo de 10 MB que escribió el TD2 ocupa solo dos:

Salida real

```
$ head -c 10000000 /dev/urandom > big.bin; sync big.bin; filefrag -v big.bin
Filesystem type is: ef53
File size of big.bin is 10000000 (2442 blocks of 4096 bytes)
ext:      logical_offset:      physical_offset: length:   expected: flags:
  0:         0..      2047:   15654912..   15656959:   2048:
  1:      2048..      2441:   15673843..   15674236:    394:   15656960: last,eof
big.bin: 2 extents found
```

Practica esto → A3.4 Del inodo a los bloques · TD2 · 1

§3 Nombres: directorios, enlaces y rutas

Un **directorio** es un archivo cuyos datos son una lista de pares *nombre* → *número de inodo*, más `.` (él mismo) y `..` (su padre). Para abrir `/home/student/notas.txt` el núcleo parte del inodo raíz (el 2 en ext4), lee el directorio, busca `home`, sigue con su inodo, y así componente por componente (*path_resolution(7)*). Cada componente exige permiso de paso (`x`) en el directorio. Linux guarda las resoluciones recientes en la *caché de dentries*.

Un **enlace duro** (`link`, `ln`) es otra entrada que apunta al **mismo** inodo: los dos nombres son igual de "originales" y `st_nlink` los cuenta. No se permiten hacia directorios (romperían el árbol) ni entre sistemas de archivos (los números de inodo son locales). Un **enlace simbólico** (`symlink`, `ln -s`) es un inodo nuevo cuyo contenido es una *ruta*; la resolución la sigue (a lo sumo 40 veces: después, `ELOOP`) y puede quedar colgando si el destino desaparece.

Salida real

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

`rm` llama a `unlink`, que borra una **entrada** y resta 1 a `st_nlink`. El núcleo libera el inodo y sus bloques solo cuando `st_nlink = 0` y ningún proceso lo tiene abierto. Mientras tanto el archivo sigue legible por su descriptor:

Salida real

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l
atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

⚠ Comprobar y luego usar: la carrera TOCTOU

`if (access(path, W_OK) == 0) fd = open(path, O_WRONLY);` tiene una ventana: entre las dos llamadas otro proceso puede reemplazar `path` por un enlace a otro archivo. Abre directamente y trata el error (`EACCES`, `ENOENT`), o usa `O_NOFOLLOW` u `openat2()` con `RESOLVE_NO_SYMLINKS`. Además, `access` comprueba con el UID *real*, no con el efectivo.

Practica esto → A3.1 Inodos y directorios

§4 Archivos abiertos: las tres tablas

`open(path, flags, mode)` resuelve la ruta, verifica permisos y devuelve el **menor descriptor libre**. Las banderas: acceso (`O_RDONLY`, `O_WRONLY`, `O_RDWR`), creación (`O_CREAT`, `O_EXCL`, `O_TRUNC`); los permisos pedidos pasan por la `umask` y estado (`O_APPEND`, `O_NONBLOCK`, `O_CLOEXEC`). glibc implementa `open` con `openat(AT_FDCWD, ...)`, que es lo que muestra `strace`.

Detrás de cada descriptor hay tres niveles:

1. **Tabla de descriptors**, una por proceso: el número (0, 1, 2, 3...) es un índice en ella.
2. **Tabla de archivos abiertos**, una para todo el sistema: cada `open` crea una entrada con el modo, el **desplazamiento** actual y un contador de referencias.
3. **Tabla de inodos en memoria**: una copia del inodo por archivo, compartida por todas las aperturas.

Por eso importa quién comparte cada entrada. Dos `open` del mismo archivo crean dos entradas con desplazamientos independientes; `dup`, `dup2` y `fork` copian el *puntero* y comparten el desplazamiento. Con `donnees.txt = abcdefgh`:

Salida real

```
$ ./alias_open
abcdabcd
```

Salida real

```
$ ./alias_dup
abcdefgh
```

Consecuencias prácticas: `./prog > out 2> out` abre `out` dos veces y la salida de error pisa a la estándar; `./prog > out 2>&1` usa `dup2` y no pierde nada. Con `O_APPEND`, cada `write` se mueve al final de forma atómica: así escriben sus registros los servidores. `close` resta una referencia; la entrada se libera cuando llega a 0.

Practica esto → A3.2 Las tres tablas · TD2 · 3

§5 Archivos especiales: FIFO, dispositivos, `/proc`

"Todo es un archivo" significa que muchas cosas que no son datos en un disco usan la misma interfaz `open` / `read` / `write` / `close`. Una **FIFO** (tubería con nombre, `mkfifo`) es un inodo sin bloques: lo que un proceso escribe, otro lo lee a través de un búfer del núcleo de 64 KiB. Sus reglas, que el chat del TD2 usa:

- `open(O_RDONLY)` espera a que alguien abra para escribir, y viceversa (con `O_NONBLOCK`, el escritor sin lector recibe `ENXIO`).
- Una escritura de hasta `PIPE_BUF` = 4096 bytes es **atómica**: no se mezcla con la de otro escritor. Una más grande sí puede mezclarse.

- `read` devuelve 0 (fin de archivo) cuando cerró el **último** escritor.
- Escribir sin ningún lector produce la señal `SIGPIPE` (o el error `EPIPE` si se ignora).

Otros archivos especiales: los **dispositivos** de caracteres y de bloques en `/dev` (creados con `mknod`), los **sockets** de dominio UNIX (unidad 8 y TD8) y los sistemas de archivos virtuales `/proc` y `/sys`, cuyos "archivos" genera el núcleo al leerlos.

Salida real

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' | ./fifo_client
chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Practica esto → A3.3 FIFO: el chat · TD2 · 2

§6 Recorrer directorios

`opendir` abre un directorio, `readdir` devuelve una `struct dirent` por entrada (o `NULL` al final) y `closedir` lo cierra. Las entradas llegan en el orden interno del sistema de archivos —en ext4, el de un índice por hash—, e incluyen `.` y `..`. De `struct dirent` interesan `d_name`, `d_ino` y `d_type`; este último ahorra un `stat`, pero puede valer `DT_UNKNOWN` en algunos sistemas de archivos, y entonces hay que llamar a `lstat`. Para recorrer un árbol, nunca se siguen los enlaces simbólicos a directorios: podrían formar ciclos.

C tds/td2/myls.c

```
static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue with the
        others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
}
```

Salida real

```
$ ./myls -lR demo
demo:
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student    3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student    5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student    0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student  4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student  4096 2026-09-30 13:12 ..
```

Practica esto → TD2 · 4 Un `ls` propio

§7 Más allá: diario, copia en escritura y `fsync`

- **Diario (*journaling*)**. Crear un archivo modifica varios bloques (mapa de bits, inodo, directorio). Si se corta la luz a mitad, el sistema queda inconsistente. ext4 y XFS escriben primero la operación en un diario y luego la aplican: al reiniciar basta con repetir o descartar lo pendiente.
- **Copia en escritura**. Btrfs y ZFS nunca sobrescriben un bloque en su lugar: escriben uno nuevo y cambian el puntero. Así obtienen instantáneas casi gratis y sumas de verificación de cada bloque.
- **Caché de páginas y `fsync`**. `write` termina cuando los datos están en la caché del núcleo, no en el disco. Un programa que necesita durabilidad (una base de datos, un editor al guardar) llama a `fsync`, y para que el *nombre* también sea durable, a `fsync` sobre el directorio.
- **FUSE**. Un sistema de archivos puede implementarse como un proceso de usuario (sshfs, rclone): el núcleo le reenvía las llamadas.